Reproducibility package

Re-estimation-verified selection of small parameter subsets for systems biology models

Mónica Miranda R., Carlos Salazar, César Martín, Adriana Aguirre (Escuela Superior Politécnica del Litoral, ESPOL).
Submitted to *Scientific Reports*.

This package contains everything needed to check the results of the paper:

- the **Python programs** of the method and of every numerical analysis (`analysis/`);
- the **results** of those analyses, one file per benchmark problem (`results/`, 16 MB);
- the **programs that produce every figure, table and Supplementary Note** of the paper from those results (`figures_tables/`);
- the **formal proofs** in Lean 4 with Mathlib: 32 modules with 309 theorems and lemmas, the hypotheses of 28 benchmark models, and machine-checked non-vacuity witnesses for the central theorems (`lean_proofs/`);
- the **translator** of the SBML models into Lean and an independent check of the translation (`lean_translation/`).

The benchmark problems themselves are downloaded from the PEtab benchmark collection (`scripts/get_benchmark.sh`). No new data from human participants were collected; some benchmark data are human-derived (for example the published viral-load measurements of the Perelson problem) and are analysed as published.

Three levels of checking

Level	What it does	Time	Needs
A	Regenerates every figure, table and Supplementary Note from the stored results (no simulation)	a few minutes	Python, LaTeX
B	Compiles all formal proofs and checks that they use no unproved assumption	20 min (many cores) to 2 h	Lean (elan)
C	Re-runs the analyses from the PEtab problems and compares with the stored results	the method (selection): minutes per problem, 1.3 h of processor time for all 34; all analyses of the paper: about 697 h of processor time (about a week on 4 cores, a day on 32 cores)	Python, the benchmark (download)

Each level is independent. A reader who only wants to see where a number comes from needs level A (it shows that every number of the paper follows from the stored results, not that those results are correct); a reader who wants to check the mathematics needs level B (it shows that every theorem is proved from its hypotheses with no unproved assumption); level C repeats the computations from the public benchmark and shows that the stored results are those the code produces. The selection of every problem, or all analyses of a few problems, can be repeated on a laptop; repeating everything for the largest problems (Chen, Isensee, Raimundez, Smith, Bachmann) needs a many-core server.

Contents

```
README.md                this manual
DATA_DICTIONARY.md       meaning of every field of the result files
requirements.txt         Python packages (exact versions); environment.yml for conda
LICENSE                  MIT (code); LICENSE-results.md: CC BY 4.0 (results)
CITATION.cff             how to cite
scripts/
  get_benchmark.sh       downloads the 35 PEtab problems used (commit fcbddf1, about 80 MB)
  compare_results.py     compares re-computed results with the stored ones
  run_second_review.sh       runs all re-analyses of the second review (A1-A6) on a many-core server
  run_identifiability_second_review.sh  runs the identifiability analyses of the second review
  run_identifiability_other_criteria.sh  profiles of the subsets of Brun, Morris, standard QR and profile
                         likelihood (the four criteria in parallel; version 1.6)
  run_test_kmin.sh         test of the minimum subset size k_min = 2: the best single parameters through
                         robustness, +-20 % admissibility, synthetic and identifiability tests (version
1.6)
  run_test_calibration_fixed.sh  admissibility at +-5 % with the calibration parameters kept at theta0,
                         all subsets of the 14 problems with calibration parameters (version 1.6)
  setup_server.sh     prepares a fresh Linux server (clone, Python environment, benchmark)
analysis/                the method and the numerical analyses (Python)
  config.py              paths and settings of the paper (delta, tau_R, tau_kappa, tau_e, seeds, ...)
  petab_problem.py       simulation of the measured outputs of a PEtab problem
  selection.py           parameter selection (stages 1 and 2, admissibility, backward elimination)
  external_validation.py  prediction of held-out experiments (real and synthetic data)
  robustness.py          errors at perturbation sizes 1-50 % (paired scenarios), patterns A-D;
                         ROBUST_INDEP=1: independent directions at every size (point B5);
                         ROBUST_SUBSETS=<folder>: other subsets; ROBUST_LEVEL=<size>: one size per process
  absolute_error.py      reproduction error in units of the noise sigma (point C1)
  single_parameter.py    single-parameter subsets (minimum size 2)
  simulator_check.py     comparison with the reference simulations of the benchmark
  timing.py              time of one simulation (computational cost)
  nested_validation.py   leakage-free nested validation (fixed parameters re-estimated on training data)
  synthetic_seeds.py     synthetic prediction experiment with 20 seeds (reduced and full model);
                         SINT_AMPLITUD=a: truth theta0 (1 + a u); SINT_SOLO_SENAL=1: only the true signal
                         y(theta*) - y(theta0) in noise units, without fits (point C3)
  baselines.py           random subsets and calibration-only re-estimation (baselines)
  qr_selection.py        selection by column-pivoted QR (same k, same validation)
  brun_morris.py         selection by Brun's collinearity index and by Morris screening
  profile_likelihood.py profile likelihood of every biological parameter; selection by profile
  criteria_tests.py      admissibility (+-5 %, +-20 %) and synthetic test of the subsets of all criteria
  identifiability_profiles.py    profile likelihood of the reduced problem (S and calibration free)
  identifiability_local.py       kappa and VIF of the sensitivities of S and the calibration parameters
  identifiability_conditions.py  quantities of the Lean theorem ProfileIdentifiability (I_j, gamma,
remainder)
  identifiability_server.py/.sh  the same profiles on a many-core Linux server (Jacobian in parallel)
  efit_nonlinear.py      nonlinear re-estimation of the final subsets of the first-order problems (A2)
  conditioning_check.py kappa and VIF of the final subsets with centred and with unit-length columns
  identifiability_box.py hypotheses of the Lean identifiability theorems over the admissible set,
                         by region radius (sampling and adversarial search)
```

```
  crauste_blowup.py     why no local reduction exists for Crauste
  prepare_criteria_subsets.py, compare_identifiability_criteria.py  identifiability of the subsets of
                        the other criteria (input of identifiability_profiles.py, summary)
  prepare_single_subsets.py, compare_single_vs_pair.py, kmin_times.py  test of k_min = 2 (best single
                        parameters, comparison with the selected pair, computing time)
  calibration_fixed_summary.py  summary of the test with the calibration parameters fixed
                        (criteria_tests.py with CT_CAL_FIXED=1 produces its results)
results/                stored results (see DATA_DICTIONARY.md)
  selection/            selected subset of each of the 34 problems
  settings/<variant>/   the same with one setting changed (15 variants, 26 problems)
  settings_extra/<variant>/  the same variants for Lang and Raimundez (three did not finish; README.md)
  external_validation/  held-out prediction, real and synthetic data
  robustness/           errors of the 15 scenarios at 7 perturbation sizes
  robustness_independent/  the same with independent directions at every size
  absolute_error/       reproduction error in units of sigma at +-5 %
  single_parameter/     single-parameter subsets
  nested_validation/    leakage-free nested validation (29 problems)
  synthetic_seeds/      synthetic experiment, 20 seeds (one file per problem and seed; RESUMEN.json)
  synthetic_seeds_a005/, synthetic_seeds_a050/   the same with truths at +-5 % and +-50 % (seeds 42-51)
  synthetic_signal_a005/, _a020/, _a050/          true signal y(theta*) - y(theta0) only (no fits)
  random_subsets/, calibration_only/   baselines
  qr_selection/, brun_morris/, profile_likelihood/   selections of the other criteria
  criteria_tests/, criteria_tests_20pc/  admissibility and synthetic test of the five criteria
  identifiability/      profiles of the reduced problem (profiles/), local check (local/) and the
                        conditions of the Lean theorem (certificado*.csv); summary.md
  timing_complete/      run time of the complete method from scratch (parallel, single core,
                        and the eight first-order problems with the corrected code)
  simulator_check/, cost/, sbml_to_lean/, crauste_blowup.json
  _before_negative_fix/  earlier results of Armistead, Raimundez and Lang, kept for traceability (not
used)
  --- second review (version 1.5) ---
  selection_fixedscen/  selection with a fixed scenario set (A5), with the monotonicity log
  selection_train_fixedscen/  selection on the training outputs only (A1)
  qr_selection_gks/     standard (Golub-Klema-Stewart) pivoted QR (A4)
  efit_nonlinear/       nonlinear e_fit of the six first-order problems, one file per scenario (A2)
  criteria_tests_fixedscen/, criteria_tests_20pc_fixedscen/  admissibility of every criterion, of
                        k = 0 and of 20 random subsets, real-data and synthetic tests (A1, A3, A4)
  criteria_tests_sint_loguniforme/, criteria_tests_sint_dispersa/  other families of truths (A6)
  conditioning_check.json  kappa and VIF, centred and unit-length, of the final subsets
  identifiability/profiles_dense/  profiles with 20 points per side
  identifiability/profiles_selection_train_fixedscen/  profiles of the training-output subsets
  identifiability/box/, identifiability/audit2/  hypotheses of the Lean theorems by region radius
  audit2_stats.json     statistics of Table 2 and Fig. 4 (written by figures_tables/audit2_stats.py)
  --- version 1.6 ---
  subsets_brun/, subsets_morris/, subsets_qr_gks/, subsets_perfil/  subsets of the other criteria
  identifiability/profiles_subsets_<criterion>/  their profiles (every point);
criteria_identifiability.json
                        (column Identifiable of Table 2);
identifiability/audit2/certificado_<criterion>.csv
  logs_other_criteria/*_summary.log  logs of that run (progress and results only)
  subsets_single/       best single parameter of the 15 problems in which one already suffices
```

```
  robustness_subsets_single/, robustness_independent_subsets_single/  its robustness
  identifiability/profiles_subsets_single/  its profiles; criteria_tests_*fixedscen/*__single__*  its
                       admissibility at +-20 % and synthetic tests
  kmin_comparison.json, kmin_summary.json, kmin_times.json  comparison with the pair and computing time
  logs_kmin/    logs of that run
  criteria_tests_calfixed/, calibration_fixed_summary.json, logs_calibration_fixed/  admissibility
                       with the calibration parameters fixed and its summary
figures_tables/        produce the figures, tables and Supplementary Notes from results/
  build_all.py         runs all of them (level A)
  comparison_*.py      figures, tables and statistics of the comparison with other criteria
  make_review_notes.py  tables and figures of the additional analyses (B5, C3, C1, settings of Lang and
Raimundez)
  identif_criteria_tables.py  identifiability of the subsets of every criterion (Suppl. Tables, Table 2
column)
  kmin_tables.py       test of k_min = 2 (Supplementary Table)
  calibration_fixed_tables.py  admissibility with the calibration parameters fixed (Supplementary Tables)
  conditioning_table.py  conditioning of the final subsets with unit-length columns (Supplementary Table)
  note_edits.py, note_edits/  editorial edits of the generated Supplementary Notes (applied
automatically)
  main_text_numbers.py      recomputes the counts quoted in the main text from results/
  make_timing_notes.py  computing time of every analysis for every problem (Suppl. Note on computing
time)
  audit2_stats.py      statistics of the comparison after the second review, Table 2
  audit2_tables.py     supplementary tables of the second review
lean_proofs/           Lean 4 + Mathlib project (see lean_proofs/README.md)
  NonVacuity.lean      witnesses: every central theorem has an instance with all hypotheses proved
  DEFINITIONS_AUDIT.md  correspondence of each Lean definition with the paper and the code
  BUILD_LOG.md         log of the clean build and of the independent check with lean4checker
lean_translation/
  sbml_to_lean.py      translator SBML -> Lean (writes lean_proofs/Models/)
  verify_translation.py independent check of the translated models against libRoadRunner
  mutation_control.py  control: a 1 % change of one number of a Lean file is detected
```

Installation

Python (3.11; tested on Linux with the versions in `requirements.txt`):

```
python3.11 -m venv .venv
source .venv/bin/activate           # Windows: .venv\Scripts\activate
pip install -r requirements.txt
```

or, with conda, `conda env create -f environment.yml && conda activate fewparams`.

Other tools (level A only): `pdflatex` (any TeX distribution, for Fig. 6) and `pdftops`, `pdftoppm` (poppler-utils, for the EPS and TIFF copies of the figures).

Lean (level B only): install elan; it installs the Lean version given in `lean_proofs/lean-toolchain` (4.23.0) automatically.

All commands below are run from the folder of this README.

Level A — figures, tables and Supplementary Notes from the stored results

```
cd figures_tables
python build_all.py
```

The output is written to `figures_tables/output/`:

- `tables/table1.tex` (Table 1), `tables/tableS1.tex`, `tables/tableS3.tex`;
- `figures/fig1_pipeline.pdf` … `fig6_architecture.pdf` (with `.eps` and `.tif` copies) and the supplementary figures `figS1_validation_scatter.pdf`, `figS_robustness_scenarios.pdf`;
- `figure3/`: the consolidated table behind Fig. 3 and a summary of the counts quoted in the text;
- `suppl_*.tex`: the Supplementary Notes that contain numbers (simulator, synthetic experiment, single parameter, settings, cost, robustness, system-by-system summary, verified models, statements of the formal theorems);
- comparison with other criteria: `figures/fig3_validation_v2.pdf` (Fig. 3), `figures/fig4_comparison.pdf` (Fig. 4), the supplementary figures `figS_profiles.pdf`, `figS_admissibility_map.pdf`, `figS_synth_seeds.pdf`, `figS_nested.pdf`, the supplementary tables `tables/tableS_*.tex` (profile likelihood, selections, real data, Wilcoxon, admissibility, McNemar, synthetic, nested validation, run times) and, in `comparison/`, the statistics and the counts quoted in the text;
- additional analyses: `tables/tableS_robust_indep.tex` and `figures/figS_robust_indep.pdf` (independent directions), `tables/tableS_signal.tex` and `figures/figS_signal.pdf` (signal and amplitudes of the synthetic experiment), `tables/tableS_abs.tex` (error in units of sigma) and `tables/tableS_settings_extra.tex` (settings of Lang and Raimundez).

Each script prints the counts that the paper quotes (for example the numbers of admissible problems for each setting), so that they can be compared with the text.

Several Supplementary Notes were edited by hand after they were generated (wording requested by the reviews, sections added later, cross-references). These edits are stored in `figures_tables/note_edits/`, one JSON file per note, and are applied by `note_edits.py` on top of the generated text, so that the notes written to `output/` are identical to those of the Supplementary Information; every number in them is still computed from `results/`. Two tables are not generated: Table 3 (what the formal verification guarantees), which is written by hand from the Lean development, and Supplementary Table S3 (parameter counts), which is read from the PEtab parameter tables of the benchmark.

To see where a particular number of the paper comes from, use the table Where each result comes from and open the JSON file of the problem (fields explained in `DATA_DICTIONARY.md`).

Level B — the formal proofs

```
cd lean_proofs
./verify.sh
```

The script (i) checks that no file contains `sorry` or `axiom`, (ii) downloads the precompiled Mathlib, (iii) compiles the 32 theory modules, the witnesses `NonVacuity.lean` and the 28 model files and (iv) reports the axioms used by every theorem (309) and by the final theorem of every model (28). It stops with an error unless every declaration is reported and only Lean's standard axioms (`propext`, `Classical.choice`, `Quot.sound`) appear. The first compilation takes about 20 minutes on many cores and 1–2 hours on a laptop (mostly the largest models, Chen and Froehlich); the log of our clean build is `lean_proofs/BUILD_LOG.md`.

An independent re-check of the compiled declarations with lean4checker (tag `v4.23.0`):

```
git clone https://github.com/leanprover/lean4checker && cd lean4checker
git checkout v4.23.0 && cp ../lean_proofs/lean-toolchain . && lake build && cd ../lean_proofs
for m in ScaledError Poda GreedySelection ProfileIdentifiability NonVacuity; do
  lake env ../lean4checker/.lake/build/bin/lean4checker $m; done      # any module; silent on success
```

`NonVacuity.lean` gives, for each central theorem, a concrete instance in which every hypothesis is proved (so that no central statement is vacuous) and, for defined properties, an instance in which they fail; `DEFINITIONS_AUDIT.md` compares each Lean definition with the paper and with the code. `lean_proofs/README.md` relates each step of the method to its theorems, and `lean_proofs/FORMULAS.md` gives the statement of every theorem in mathematical notation.

The model files can be regenerated from the SBML models and checked independently (needs the benchmark, see level C):

```
cd lean_translation
python sbml_to_lean.py               # writes lean_proofs/Models/ (identical files) and
        results/sbml_to_lean/
python verify_translation.py         # largest relative difference with libRoadRunner: about 1e-12
python mutation_control.py           # a 1 % change of one number is detected (difference about 1e-2)
```

Level C — re-running the analyses

1. Download the benchmark problems (35 problems at commit `fcbddf1`, about 80 MB, needs `git`):

   ```
   bash scripts/get_benchmark.sh
   ```

   (Another copy can be used by setting `PETAB_BENCHMARK=/path/to/Benchmark-Models-PEtab/problems`.)

2. Choose where to write the new results, so that the stored ones are kept for comparison:

   ```
   export REPRO_RESULTS=$PWD/results_rerun      # Windows PowerShell:
          $env:REPRO_RESULTS="$PWD\results_rerun"
   export NPROC=4                               # processes for the problems that run in parallel
   cd analysis
   ```

3. Run the steps in this order (each step uses the selection of step b). Any step can be limited to some problems by giving their names, e.g. `python selection.py Boehm_JProteomeRes2014`.

Step	Command	Paper	Time on 4 cores
a	`python simulator_check.py`	Supplementary Table S9	minutes
b	`python selection.py`	Fig. 2, Table 1, Suppl. Table S1	minutes, except Chen (~2 h), Lang, Raimundez
c	`for v in delta_0.001 delta_0.05 tolerances_1e-9_1e-12 tolerances_1e-6_1e-8 perturbation_0.01 perturbation_0.10 tauR_0.80 tauR_0.85 tauR_0.95 tauK_5 tauK_30 tauE_0.10 tauE_0.20 tauE_0.30 tauE_0.50; do python selection.py --variant $v; done`	Fig. 5, Suppl. Table S14	0.5–1 h per variant
d	`python external_validation.py`	Fig. 3, Suppl. Fig. S1, Suppl. Table S12	~6 h (Chen and Raimundez take most)
e	`python robustness.py`	Fig. 4, Suppl. Tables S10–S11, Fig. S2	a few hours
f	`python single_parameter.py`	Suppl. Table S13	~1 h
g	`python crauste_blowup.py`	Results (Crauste)	minutes
h	`python timing.py --write Boehm_JProteomeRes2014 Chen_MSB2009 ...`	Suppl. Table S15	minutes (times depend on the machine)
j	`python nested_validation.py` (Chen: `VAL_PMAX=200 JAC_NPROC=110 REPRO_RTOL=1e-6 REPRO_ATOL=1e-8 python nested_validation.py Chen_MSB2009`)	Suppl. Note nested validation	hours; Chen ~10 min on 110 cores
k	`SINT_SEEDS=42-61 python synthetic_seeds.py`	Fig. 3b, Suppl. Note synthetic experiment	~1 h on 100 cores
l	`python baselines.py`	Table 2, Suppl. Note comparison on real data	~1 h
m	`python qr_selection.py`	Table 2, Suppl. Note pivoted QR	minutes
n	`PL_PMAX=60 NPROC=120 python profile_likelihood.py`	Fig. 4, Table 2, Suppl. Note profile likelihood	~4 h on 120 cores (Isensee most)
o	`python brun_morris.py` (after m and n)	Table 2, Suppl. Note Brun and Morris	minutes
p	`python criteria_tests.py`, then `SOLO_SINT=1 SINT_SEMILLAS=47,...,61 python criteria_tests.py` and `ADM_NIVEL=0.20 python criteria_tests.py` (after l-o)	Fig. 4a-b, Table 2, Suppl. Notes admissibility and synthetic comparison	~1 h on 100 cores
q	`REPRO_RESULTS=$PWD/../results_timing NPROC=64 python selection.py` (and with `NPROC=1`)	Fig. 4d, Suppl. Note cost	the run time is the measurement
r	`python identifiability_local.py`, `NPROC=8 python identifiability_profiles.py` (Bachmann, Chen, Isensee, Lang, Raimundez and Smith: `bash identifiability_server.sh` on a server), then `python identifiability_conditions.py` (`CERT_PAR=256` to run the simulations of one problem in parallel)	Results (identifiability), Suppl. Note identifiability	~10 min on 8 cores; ~3 h on 256 cores for the six large problems

Step	Command	Paper	Time on 4 cores
s	`NCORES=88 bash scripts/run_second_review.sh` (second review: A5 fixed scenarios, A1 selection on the training outputs, A2 nonlinear check, A4 standard QR, A3 null controls, A6 other families of truths), then `python conditioning_check.py`	Table 2, Fig. 4, Methods, Suppl. Notes *Nonlinear check*, *Selection on the training outputs, null controls and standard pivoted QR*, *Conditioning with unit-length columns*	~6 h on 88 cores (Chen in A2 longest)
t	`NPROC=30 bash scripts/run_identifiability_second_review.sh` (after r and s)	Results and Suppl. Note *Identifiability over the admissible set*	~5 h on 88 cores
u	`NPROC=128 bash scripts/run_identifiability_other_criteria.sh` (after r and o-p)	Results *Identifiable and sufficient*, Table 2 (column Identifiable), Suppl. Note *Identifiability of the subsets of other criteria*	~2 h on 128 cores (41 h of processor time)
v	`NPROC=88 bash scripts/run_test_kmin.sh`, then `python kmin_times.py` (after f, e and r)	Discussion (limitation iii), Methods, Suppl. Note *Minimum subset size: one parameter or two*	~30 min on 88 cores (5.7 h of processor time)
w	`NPROC=90 bash scripts/run_test_calibration_fixed.sh` (after s)	Results (comparison), Discussion, Methods *Calibration parameters fixed*, Suppl. Note *Admissibility with the calibration parameters fixed*	34 min on 90 cores (16.7 h of processor time)

Steps j, n, p, s, t, u, v and w distribute the work over many processes and use `fork` (Linux). The stored results of steps j-q were computed on a 128-core server shared by several computations; their run times (`segundos`) may therefore be inflated by resource contention. In step v the robustness of Smith and Schwen was split into one process per perturbation size (`ROBUST_LEVEL`) during the run; `kmin_times.py` takes their time from the checkpoint files.

4. Compare each folder with the stored results:

```
cd ..
python scripts/compare_results.py results_rerun/selection results/selection
python scripts/compare_results.py results_rerun/external_validation results/external_validation
...
```

Running times and checkpoint bookkeeping are ignored; numbers are compared with a relative tolerance of 1e-6 (`--rtol` to change it).

5. Optionally, regenerate the figures and tables from the new results: `REPRO_RESULTS=$PWD/results_rerun python figures_tables/build_all.py`.

What to expect. The method is deterministic (fixed seeds), and for the small and medium problems the re-computed results agree with the stored ones to the last digits. Small differences can appear (i) in the problems whose simulations occasionally fail to integrate, so that a scenario is valid in one run and not in another (Chen: 8 to 13 of 15 scenarios per size); (ii) in the fits of the external validation, which use the result of the previous optimisation when a long fit is resumed from a checkpoint; and (iii) with other versions of libRoadRunner/SUNDIALS or of NumPy/SciPy. Running times (`segundos`, `t_sim`) depend on the machine. The comparison with other criteria (steps j-q) was computed with libRoadRunner 2.10.0; criteria that choose between

nearly tied candidates can change their choice with another simulator version (for example, pivoted QR in Boehm selects `k_exp_hetero` instead of `k_phos` with libRoadRunner 2.9.2), and single local fits (one random draw of the baselines) can end in slightly different optima.

Large problems. For the eight problems in `config.FIRST_ORDER` (Alkan, Bachmann, Beer, Chen, Isensee, Lang, Lucarelli, Raimundez) the re-estimation error is computed to first order, as stated in the Methods; their sensitivity matrices are computed in parallel (`NPROC` processes), kept on disk in single precision in `results_rerun/cache/` and the computation resumes from there if interrupted. Froehlich (4,231 parameters, 9,169 experimental conditions) is not analysed on its measured outputs (each evaluation needs about 9,000 simulations); only the time of one simulation of its state trajectories is used (cost note), and its model is verified in Lean.

Where each result comes from

Paper	Program (`analysis/`)	Results (`results/`)	Figure/table program (`figures_tables/`)
Fig. 1 (method)	—	—	`make_figures.py` (`fig1`)
Fig. 2, Table 1, Suppl. Table S1; Results, numbers of selected parameters, levels of guarantee	`selection.py`	`selection/`	`make_figures.py` (`fig2`), `make_tables.py`
Fig. 3a, Suppl. Fig. S1; Results, held-out prediction	`external_validation.py`	`external_validation/`	`make_figure3.py`, `comparison_figures.py`
Fig. 3b; synthetic experiment with 20 seeds (Supplementary Note)	`synthetic_seeds.py`	`synthetic_seeds/`	`comparison_figures.py`, `comparison_tables.py`
Leakage-free nested validation (Results; Supplementary Note)	`nested_validation.py`	`nested_validation/`	`comparison_tables.py`, `comparison_suppl_figures.py`
Comparison with other criteria: Table 2, Fig. 4 and the Supplementary Notes on profile likelihood, Brun and Morris, real data, admissibility and synthetic comparison	`qr_selection.py`, `baselines.py`, `brun_morris.py`, `profile_likelihood.py`, `criteria_tests.py`	`qr_selection/`, `random_subsets/`, `calibration_only/`, `brun_morris/`, `profile_likelihood/`, `criteria_tests/`, `criteria_tests_20pc/`	`comparison_figures.py`, `comparison_tables.py`, `comparison_statistics.py`, `comparison_summary.py`, `comparison_suppl_figures.py`
Identifiability of the selected parameters (Results; Supplementary Note and Table)	`identifiability_local.py`, `identifiability_profiles.py`, `identifiability_server.py`, `identifiability_conditions.py`	`identifiability/`	`identifiability_table.py`, `identifiability_details.py`
Run time of the complete method (Results; Fig. 4d; Supplementary Note cost)	`selection.py` run from scratch	`timing_complete/`	`comparison_tables.py`

Paper	Program (`analysis/`)	Results (`results/`)	Figure/table program (`figures_tables/`)
Fig. 5, Suppl. Tables S10–S11, Suppl. Fig. S2; patterns A–D, pass rates	`robustness.py`	`robustness/`	`make_figures.py` (`fig4`), `make_robustness_note.py`
Supplementary Note on the method settings (figure and table)	`selection.py --variant`	`settings/`	`make_fig5_settings.py`, `make_extra_notes.py`
Single-parameter subsets (Results; Suppl. Table S13)	`single_parameter.py`	`single_parameter/`	`make_extra_notes.py`
Robustness with independent directions (Results, Robustness; Supplementary Note on robustness, table and figure)	`robustness.py` with `ROBUST_INDEP=1`	`robustness_independent/`	`make_review_notes.py`
Signal and amplitudes of the synthetic experiment (Results; Supplementary Note on the synthetic experiment, table and figure)	`synthetic_seeds.py` with `SINT_SOLO_SENAL=1` and `SINT_AMPLITUD=0.05/0.50`	`synthetic_signal_a005/`, `_a020/`, `_a050/`, `synthetic_seeds_a005/`, `_a050/`	`make_review_notes.py`
Error in units of the noise (Results, settings; Supplementary Note on the method settings)	`absolute_error.py`	`absolute_error/`	`make_review_notes.py`
Random scenarios, large problems and the greedy search in Lang (Supplementary Note *Random scenarios and large problems*, settings of Lang and Raimundez)	`selection.py --variant`	`settings_extra/`	`make_review_notes.py`
Computing time of every analysis (Results, cost; Supplementary Note *Computing time of the analyses*)	all programs (field `segundos`)	all of the above	`make_timing_notes.py`
Second review: Table 2, Fig. 4 (null controls, standard QR, training-output selection, three families of truths)	`selection.py` (`REPRO_OUTPUTS=train`, `REPRO_FIXED_SCEN=1`), `qr_selection.py` (`QR_VARIANT=gks`), `criteria_tests.py` (`CT_METODOS`, `SINT_FAMILIA`)	`selection_*fixedscen/`, `qr_selection_gks/`, `criteria_tests_*fixedscen/`, `criteria_tests_sint_*/`	`audit2_stats.py`, `comparison_figures.py`, `audit2_tables.py`
Nonlinear check of the first-order problems (Results, Methods; Suppl. Note)	`efit_nonlinear.py`	`efit_nonlinear/`	`audit2_tables.py`
Conditioning with unit-length columns (Methods; Suppl. Note)	`conditioning_check.py`	`conditioning_check.json`	—
Identifiability over the admissible set, dense profiles, profiles of the training-output subsets (Results; Suppl. Note)	`identifiability_box.py`, `identifiability_profiles.py` (`ID_DENSE=1`, `ID_SUBSETS=...`)	`identifiability/box/`, `identifiability/audit2/`, `identifiability/profiles_dense/`, `identifiability/profiles_selection_train_fixedscen/`	`audit2_tables.py`

Paper	Program (`analysis/`)	Results (`results/`)	Figure/table program (`figures_tables/`)
Identifiability of the subsets of the other criteria (Results *Identifiable and sufficient*; Table 2, column Identifiable; Suppl. Note and Tables)	`prepare_criteria_subsets.py`, `identifiability_profiles.py` (`ID_SUBSETS=subsets_<criterion>`), `identifiability_conditions.py`	`subsets_*/`, `identifiability/profiles_subsets_*/`, `identifiability/criteria_identifiability.json`	`identif_criteria_tables.py`, `audit2_stats.py`
Minimum subset size k_min = 2 (Discussion, limitation iii; Methods; Suppl. Note *Minimum subset size* and Table)	`prepare_single_subsets.py`, `robustness.py` (`ROBUST_SUBSETS=subsets_single`), `criteria_tests.py` (`CT_METODOS=single`), `identifiability_profiles.py`, `compare_single_vs_pair.py`, `kmin_times.py`	`subsets_single/`, `robustness*_subsets_single/`, `identifiability/profiles_subsets_single/`, `kmin_comparison.json`, `kmin_times.json`	`kmin_tables.py`
Admissibility with the calibration parameters fixed (Results, Discussion, Methods; Suppl. Note and Tables)	`criteria_tests.py` (`CT_CAL_FIXED=1`), `calibration_fixed_summary.py`	`criteria_tests_calfixed/`, `calibration_fixed_summary.json`	`calibration_fixed_tables.py`
Counts quoted in the main text	—	all of the above	`main_text_numbers.py`
Crauste (Results, non-admissible problems)	`crauste_blowup.py`	`crauste_blowup.json`	—
Computational cost (Results; Suppl. Table S15)	`timing.py`	`cost/`	`make_extra_notes.py`
Simulator check (Methods; Suppl. Table S9)	`simulator_check.py`	`simulator_check/`	`make_extra_notes.py`
Fig. 6, Table 2; Suppl. Tables S3–S6 (formal statements)	`lean_proofs/`	—	`fig6_architecture.tex`, `make_tableS3.py`, `make_formulas.py`
Verified models, translation fidelity (Methods; Suppl. Tables S7–S8)	`lean_translation/`	`sbml_to_lean/`	`make_models.py`
System-by-system summary (Supplementary Note)	all of the above	all of the above	`make_systems_note.py`

Supplementary Tables S2 (notation) and S4 (scope of each guarantee) are text of the Supplementary Information and are not generated.

Notes

2. **Settings of the method.** All thresholds and seeds are in `analysis/config.py` and are those of the Methods: delta = 1 %, eta = 10 %, tau_R = 0.89, tau_kappa = tau_VIF = 10, tau_e = 0.436 (c = 0.9), k_min = 2, 15 scenarios at +-5 % with seed 42.

2. **Language of the result files.** Field names in `results/` are those written by the original programs (Spanish); `DATA_DICTIONARY.md` translates them. The programs of this package write the same fields, so new and stored results can be compared directly. For the same reason a few file and folder names read by the programs keep their original Spanish names: `certificado*.csv` (conditions of the Lean identifiability theorem),

`RESUMEN.json`/`RESUMEN.md` (summaries), `minimo.npz` (optimum of a profile), the criterion key `perfil` (profile likelihood, e.g. `subsets_perfil/`) and the families of synthetic truths `loguniforme` and `dispersa` (sparse). All other files, scripts and folders have English names.

- **Simulator.** libRoadRunner (CVODE, relative and absolute tolerances 1e-7 and 1e-10). Measurements at steady state and pre-equilibration use t = 1e5. If the integrator fails, the simulation is repeated piecewise with the discontinuities of the condition as breakpoints (Isensee).

- **Smith.** Its simulation time in `cost/` was measured on a second machine and scaled by 1.55 (field `nota`); all other times were measured on the same four-core server.

- **Generated and edited text.** Every table of the paper and of the Supplementary Information is regenerated identically by level A. Some Supplementary Notes (`suppl_*.tex`) were edited by hand after being generated (wording and cross-references, e.g. figure numbers), so their regenerated text can differ in wording from the submitted version; the numbers come from the same results. The Supplementary Note on the synthetic experiment is the extended 20-seed version written for the paper; `make_extra_notes.py` writes the first-seed version as `suppl_synthetic_seed42.tex`, and the tables of the 20-seed note are generated by `comparison_tables.py` and `make_review_notes.py`.

Changes in version 1.6

- **Identifiability of the subsets of the other criteria.** The parameters chosen by Brun, Morris, standard pivoted QR and profile likelihood were profiled with the procedure of our subsets (`scripts/run_identifiability_other_criteria.sh`, 128-core server, about 2 h, 41 h of processor time). Over the 27 common problems, 86 % of our selected parameters are identifiable (Brun 85 %, profile likelihood 86 %, Morris 80 %, QR 76 %); identifiable and admissible: ours 21, Brun 18, Morris 14, profile likelihood 14, QR 7 (Holm-adjusted McNemar P = 0.38, 0.047, 0.078, 0.002). New column in Table 2, new Results section, new Supplementary Note.
- **Test of the minimum subset size k_min = 2.** In the 15 problems in which one parameter already suffices, the best single parameter was put through the tests of the selected pair (`scripts/run_test_kmin.sh`, 88-core server, 27 min, 5.7 h of processor time). It is as identifiable and predictive, but fails in more individual scenarios (pass rate 88 % against 94 %, P = 0.012) and degrades more for large changes (median e_fit at +-50 % 18.7 % against 10.9 %, P < 0.001): k_min = 2 is a safety margin.
- **Admissibility with the calibration parameters fixed** (pre-submission review). The scenarios perturb the calibration parameters too, which are then re-estimated, so part of the change can be undone by them (a median of 30 % of the output change, up to 93 % in Laske). With them kept at theta0 in the 14 problems that have them (`scripts/run_test_calibration_fixed.sh`, 90 cores, 34 min, 16.7 h of processor time), our subsets remained admissible in 13 (Bachmann 44.1 %), calibration-only re-estimation fell from 9 to 6 and the admissible random draws from 220 to 192 of 280; in six problems almost any subset still passes.
- **Computing time**: all analyses together 697 h of processor time; timings on shared servers are described as possibly inflated by resource contention rather than as upper bounds.
- **Title** of the paper and of the package: *Re-estimation-verified selection of small parameter subsets for systems biology models*.

Changes in version 1.5 (second review)

- **Fixed scenario set** (`REPRO_FIXED_SCEN=1`): a scenario in which the simulation with the subset fails counts as not reproduced instead of being discarded, and the first-order `e_fit` is `min(e_rel, projection)` scenario by scenario. No subset changed; the monotonicity log (field `monotonia`) found no violation among 660 nested

pairs.

- **Selection on the training outputs** (`REPRO_OUTPUTS=train`), **standard pivoted QR** (`QR_VARIANT=gks`), **null controls** (k = 0 and 20 random subsets), **other families of synthetic truths** (`SINT_FAMILIA=loguniforme|dispersa`), **nonlinear check** of the first-order problems (`efit_nonlinear.py`) and **conditioning with unit-length columns** (`conditioning_check.py`).
- **Identifiability**: dense profiles (`ID_DENSE=1`), profiles of other subsets (`ID_SUBSETS`) and the hypotheses of the Lean theorems over the admissible set (`identifiability_box.py`); two new Lean theorems with the remainder bound only on the tail of the profile (`practically_identifiable_grad_tail`, `practically_identifiable_rel_tail`; 309 theorems).
- **Social cognitive theory example removed.** The paper no longer analyses the step-count data of the participant of the earlier study that motivated the work (it only cites that study), so the SCT programs, their stored result and the corresponding Supplementary Note are no longer part of the package.
- **Lean verification**: `NonVacuity.lean`, `DEFINITIONS_AUDIT.md`; `verify.sh` now fails unless all 309 theorems and 28 model theorems are reported with standard axioms (the model check failed silently before because `AllModels` was not a build target) and works when re-run; clean build and lean4checker recorded in `BUILD_LOG.md`.
- **Level A on Windows**: missing or incompatible poppler only produces a warning (EPS/TIFF copies); UTF-8 output is forced.

Changes in version 1.4

- **Computing time.** `figures_tables/make_timing_notes.py` (run by `build_all.py`) collects the elapsed time recorded by every analysis for every problem (field `segundos` of the result files) and writes the four tables of the Supplementary Note *Computing time of the analyses*: the method, the validation and identifiability analyses, the comparison with the other criteria and profile likelihood, and a summary per analysis. Its purpose is to show that selecting and verifying a subset is affordable: less than ten minutes of selection in 32 of the 34 problems and a median of 34 s for the identifiability of the selected parameters, against a full-model profile likelihood that could not be completed for the large problems.

Changes in version 1.3

- **Scenarios.** The random scenarios are `theta0 * (1 + epsilon * u)` without any clipping: parameters with negative or very small nominal values keep their sign and their relative change (previous versions clipped the perturbed values at 1e-12, which affected Armistead, Raimundez and Lang). Armistead now selects {k_d, k00} and Raimundez {kdeg_membran__MKN1, ka_MAPK__MKN1}; Lang has no admissible subset with the default settings (31 of 34 problems admissible). All analyses of these three problems were recomputed; the earlier files are in `results/_before_negative_fix/`.
- **Additional analyses.** Robustness with independent directions (`ROBUST_INDEP=1`, `results/robustness_independent/`), the true signal and other amplitudes of the synthetic experiment (`SINT_SOLO_SENAL`, `SINT_AMPLITUD`, `results/synthetic_signal_*`, `results/synthetic_seeds_a005`, `_a050`), the error in units of the noise (`absolute_error.py`, `results/absolute_error/`) and the settings variants of Lang and Raimundez (`results/settings_extra/`); tables and figures by `figures_tables/make_review_notes.py`.
- **Caches.** For problems with more than 200 parameters each settings variant keeps its own cache of J, of the stage-2 ordering and of the e_fit already computed.
- **Checkpoints.** Analyses that resume from checkpoints (external and nested validation, pivoted QR, random subsets, calibration only) identify them by problem and number of free parameters; delete the results and checkpoints of a problem before re-running it with a different subset.
- `figures_tables/main_text_numbers.py` recomputes the counts quoted in the main text.

Changes in version 1.2

- **Identifiability of the selected parameters.** New analyses `identifiability_*.py`, results `results/identifiability/`, Lean module `lean_proofs/ProfileIdentifiability.lean` (13 theorems) and two theorems in `Poda.lean` (inclusion-minimality for a monotone criterion); the formalisation has 32 modules and 304 declarations.
- **One run for all real-data numbers.** `figures_tables/common.load_validation()` takes the test chi2/n of the reduced model, the refitted full model and theta0 from `results/nested_validation` (falling back to `external_validation` where that run has no value), for Table 1, Fig. 3 and the system summaries, which previously used `external_validation`.
- **Admissibility comparison.** Bertozzi is excluded (`common.EXCLUDE_ADMISSIBILITY`): our method selected its parameters on all measured outputs, which include the test conditions, whereas the other criteria selected on the training outputs.
- `figure_qr_scatter.py` regenerates the pivoted-QR scatter plot (Supplementary Figure).

Corrections in version 1.1

- **First-order re-estimation (`analysis/selection.py`).** With biological-only selection, version 1.0 computed the columns of the sensitivity matrix only for the biological parameters, so the first-order re-estimation used for the eight problems of `config.FIRST_ORDER` did not re-estimate the calibration parameters. The stored results were not affected (they had been computed with a cached matrix that contained those columns), but a run from scratch gave different subsets for Bachmann, Chen, Isensee, Lang and Raimundez. The columns of the calibration parameters are now computed as well; run from scratch, the program reproduces the stored subsets and `eajuste_med` of all eight problems (`results/timing_complete/parallel_corrected_code/`).
- **Attribution.** The pivoted-QR control was attributed to Brun et al. in version 1.0; it is now `qr_selection.py`, and Brun's collinearity index is implemented in `brun_morris.py`.

License and citation

Code: MIT (`LICENSE`). Results: CC BY 4.0 (`LICENSE-results.md`). Please cite the paper and this package (`CITATION.cff`). Contact: Mónica Miranda R., mmmirand@espol.edu.ec.